CSE 464: Software QA and Testing
Project Part 3 - README

Dev Janyani | CSE-464-2026-djanyani

GitHub Repository

https://github.com/devjanyani/CSE-464-2026-djanyani

Pull Request:

https://github.com/devjanyani/CSE-464-2026-djanyani/pull/1

How to Build and Test

Clone the repository, checkout the refactor branch, and run:

```
git checkout refactor
cd graph-parser
mvn package
```

This will compile the code, run all unit tests, and package the JAR. Java JDK 25 and Maven are required.

How to Run BFS/DFS/Random Walk Search:

Run the Main class from IntelliJ or use:

```
Graph g = DotParser.parseGraph("input/sample.dot");

// BFS search
Path bfs = g.graphSearch("a", "h", Algorithm.BFS);

// DFS search
Path dfs = g.graphSearch("a", "h", Algorithm.DFS);

// Random Walk search
Path rw = g.graphSearch("a", "c", Algorithm.RANDOM);
```

Feature 1: Code Refactoring (5 Refactorings)

Refactoring 1: Extract Method - getSortedNodes()

Extracted duplicated sorted-nodes logic from toString() and buildDotString() into a reusable helper method getSortedNodes(). This eliminates code duplication and improves maintainability.

Commit: daf3ef0

Refactoring 2: Clean Up Redundant Imports

Removed redundant explicit imports (java.util.Stack, java.util.Queue, java.util.LinkedList) since java.util.* already covers them. Reduces clutter in the file header.

Commit: 05b8543

Refactoring 3: Extract Method - getSortedEdgePairs()

Extracted duplicated edge-sorting iteration logic from getEdges() and buildDotString() into a reusable helper method getSortedEdgePairs(). Eliminates code duplication.

Commit: e73bd28

Refactoring 4: Encapsulate Field - getNeighbors()

Added a public getNeighbors(String node) method to encapsulate access to the internal adjacency map. This improves encapsulation and allows external classes (like search strategies) to access neighbor data without exposing the raw map.

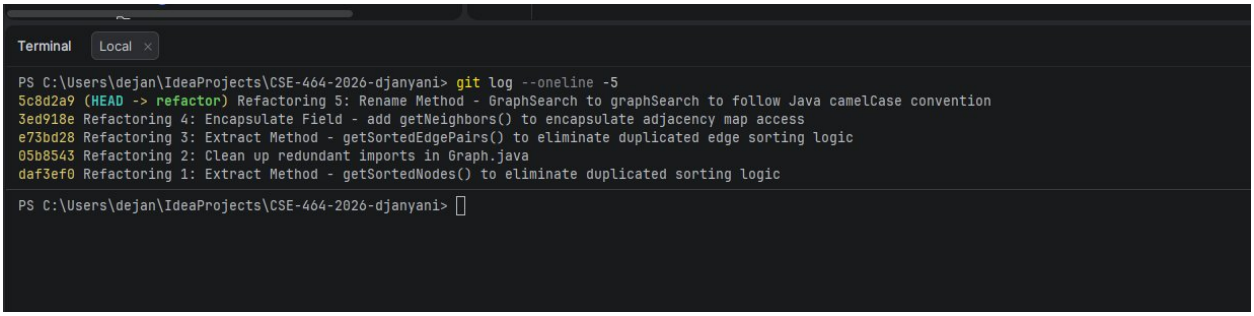
Commit: 3ed918e

Refactoring 5: Rename Method - graphSearch

Renamed GraphSearch() to graphSearch() to follow Java naming conventions (methods should use camelCase, not PascalCase). Updated all call sites in Main.java accordingly.

Commit: 5c8d2a9

Screenshot - Refactoring Commits:



Screenshot - All Tests Passing After Refactoring:

```
Terminal   Local

PS C:\Users\dejan\IdeaProjects\CSE-464-2026-djanyani\graph-parser> mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< edu.asu.cse464:graph-parser >--------------------
[INFO] Building graph-parser 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ graph-parser ---
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ graph-parser ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ graph-parser ---
[INFO] skip non existing resourceDirectory C:\Users\dejan\IdeaProjects\CSE-464-2026-djanyani\graph-parser\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ graph-parser ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ graph-parser ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.asu.cse464.DotParserTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in edu.asu.cse464.DotParserTest
[INFO] Running edu.asu.cse464.GraphTest
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in edu.asu.cse464.GraphTest
[INFO] Running edu.asu.cse464.GraphvizExportTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.500 s -- in edu.asu.cse464.GraphvizExportTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ graph-parser ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.115 s
[INFO] Finished at: 2026-04-28T01:03:06-07:00
[INFO] ------------------------------------------------------------------------

PS C:\Users\dejan\IdeaProjects\CSE-464-2026-djanyani\graph-parser>
```

Feature 2: Template Pattern for BFS and DFS

The Template Pattern was applied to extract the common structure of graph search algorithms into an abstract base class **GraphSearchStrategy**. This class defines the skeleton of the search algorithm with abstract methods that subclasses implement differently:

initFrontier(String src) - Initializes the data structure used to track nodes to visit. BFS uses a Queue, DFS uses a Stack.

isFrontierEmpty() - Checks if there are more nodes to explore.

getNextNode() - Retrieves the next node to visit. BFS polls from the queue (FIFO), DFS pops from the stack (LIFO).

addToFrontier(String node) - Adds a neighbor to the frontier.

Class Hierarchy:

```
GraphSearchStrategy (abstract base class)
  +-- BFSStrategy    (uses Queue - FIFO)
  +-- DFSStrategy    (uses Stack - LIFO)
  +-- RandomWalkStrategy (uses random neighbor selection)
```

Each subclass overrides the **search()** method to implement algorithm-specific behavior (e.g., BFS marks visited on enqueue, DFS marks visited on pop, Random Walk picks a random neighbor). The common operations — checking node existence, building and printing paths — are shared across all.

Commit:

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/63fc4f9

Feature 3: Strategy Pattern for Algorithm Selection

The Strategy Pattern was applied so that the **Algorithm** enum directly maps to the corresponding search strategy. The **getStrategy()** method in the enum returns the appropriate concrete strategy class:

```
public enum Algorithm {
    BFS,
    DFS,
    RANDOM;

    public GraphSearchStrategy getStrategy() {
        return switch (this) {
            case BFS -> new BFSStrategy();
            case DFS -> new DFSStrategy();
            case RANDOM -> new RandomWalkStrategy();
        };
    }
}
```

The **graphSearch()** method in Graph.java is now simplified to a single line that delegates to the strategy:

```
public Path graphSearch(String src, String dst, Algorithm algo) {
    GraphSearchStrategy strategy = algo.getStrategy();
    return strategy.search(src, dst, this);
}
```

This decouples the algorithm selection from the Graph class. Adding a new search algorithm only requires creating a new strategy subclass and adding an enum value — no changes to Graph.java are needed.

Commit:

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/7caba36

Feature 4: Random Walk Search Algorithm

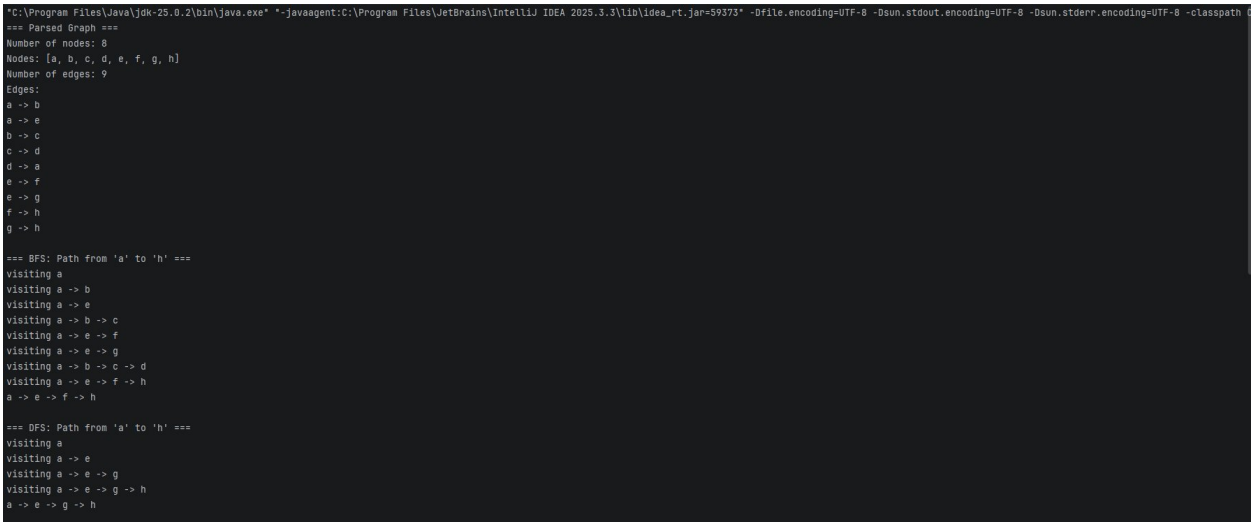
A Random Walk search algorithm was implemented using both the Template and Strategy patterns. The **RandomWalkStrategy** class extends **GraphSearchStrategy** and overrides the search method to randomly select the next neighbor to visit at each step.

At each step, the algorithm picks a random neighbor of the current node. If it reaches a dead-end node (no outgoing edges), it restarts from the source node. This ensures the algorithm eventually finds a path if one exists. A maximum step limit of 100 prevents infinite loops.

Example Usage:

```
Graph g = DotParser.parseGraph("input/sample.dot");
Path path = g.graphSearch("a", "c", Algorithm.RANDOM);
System.out.println(path);
```

Screenshot - BFS and DFS Output with Traversal:



```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=59373" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C
=== Parsed Graph ===
Number of nodes: 8
Nodes: [a, b, c, d, e, f, g, h]
Number of edges: 9
Edges:
a -> b
a -> e
b -> c
c -> d
d -> a
e -> f
e -> g
f -> h
g -> h

=== BFS: Path from 'a' to 'h' ===
visiting a
visiting a -> b
visiting a -> e
visiting a -> b -> c
visiting a -> e -> f
visiting a -> e -> g
visiting a -> b -> c -> d
visiting a -> e -> f -> h
a -> e -> f -> h

=== DFS: Path from 'a' to 'h' ===
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
a -> e -> g -> h
```

Screenshot - Random Walk Output (5 runs, a to c):

```
=== Random Walk Searches from 'a' to 'c' (5 runs) ===

random testing 1
visiting a
visiting a -> b
visiting a -> b -> c
a -> b -> c

random testing 2
visiting a
visiting a -> b
visiting a -> b -> c
a -> b -> c

random testing 3
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
visiting a
visiting a -> e
visiting a -> e -> g
visiting a -> e -> g -> h
visiting a
visiting a -> b
visiting a -> b -> c
a -> b -> c
```

```
random testing 4
visiting a
visiting a -> b
visiting a -> b -> c
a -> b -> c

random testing 5
visiting a
visiting a -> b
visiting a -> b -> c
a -> b -> c

Process finished with exit code 0
```

The output shows randomness in the search process. Some runs find the path directly (a -> b -> c), while others explore different branches (e.g., a -> e -> g -> h) and restart before eventually finding the path. Each run may produce a different traversal order.

Commit:

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/456e303

Feature 5: Pull Request and Self-Review

A Pull Request was created from the **refactor** branch to **main**. Five review comments were added as a self-review, covering the Template Pattern, Strategy Pattern, BFS/DFS data structure choices, and the Random Walk restart mechanism. Each review comment was responded to with an acknowledgment and explanation.

Pull Request Link:

https://github.com/devjanyani/CSE-464-2026-djanyani/pull/1

Summary of All Commits (refactor branch)

daf3ef0: Refactoring 1: Extract Method - getSortedNodes()

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/daf3ef0

05b8543: Refactoring 2: Clean up redundant imports

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/05b8543

e73bd28: Refactoring 3: Extract Method - getSortedEdgePairs()

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/e73bd28

3ed918e: Refactoring 4: Encapsulate Field - getNeighbors()

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/3ed918e

5c8d2a9: Refactoring 5: Rename Method - graphSearch

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/5c8d2a9

63fc4f9: Apply Template Pattern for BFS/DFS

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/63fc4f9

7caba36: Apply Strategy Pattern - Algorithm enum selects strategy

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/7caba36

456e303: Add Random Walk Search algorithm

https://github.com/devjanyani/CSE-464-2026-djanyani/commit/456e303

Branch:

refactor branch: https://github.com/devjanyani/CSE-464-2026-djanyani/tree/refactor